今天的学习内容：函数到面向对象

📖 学习目标

从函数基础到面向对象编程，全面掌握Python核心语法，为数据分析打下坚实基础。

上午：函数进阶与文件操作（6小时）

一、函数参数详解

📚 内容讲解

```python
# 1. 位置参数 - 按顺序传递
def add(x, y):
    return x + y

add(3, 5)  # 8

# 2. 关键字参数 - 按名字传递
add(x=3, y=5)
add(y=5, x=3)  # 顺序无关

# 3. 默认参数 - 有默认值
def greet(name, greeting='你好'):
    return f'{greeting}，{name}'

greet('张三')  # 你好，张三
greet('李四', '早上好')  # 早上好，李四

# 4. 可变参数 - *args（元组）
def sum_all(*args):
    return sum(args)

sum_all(1, 2, 3, 4, 5)  # 15

# 5. 关键字可变参数 - **kwargs（字典）
def print_info(**kwargs):
    for key, value in kwargs.items():
        print(f'{key}: {value}')
```

```
print_info(name='张三', age=25, city='北京')
```

🎯 应用场景

- 位置参数：简单的函数调用
- 关键字参数：参数多、可读性要求高
- 默认参数：有常用默认值的参数
- `*args`：处理不确定数量的数据（如批量求和、统计）
- `**kwargs`：处理配置信息、动态参数

💡 为什么重要（数据分析视角）

数据分析函数经常需要处理**任意数量的列**或**灵活的配置选项**，可变参数非常实用。

🔧 实战示例

```
# 数据分析场景：计算多列统计量
def calculate_stats(*columns):
    """计算多列的统计量"""
    result = {}
    for i, col in enumerate(columns):
        result[f'col{i+1}'] = {
            'mean': sum(col) / len(col),
            'max': max(col),
            'min': min(col)
        }
    return result

# 使用
col1 = [1, 2, 3, 4, 5]
col2 = [10, 20, 30, 40, 50]
col3 = [100, 200, 300, 400, 500]

stats = calculate_stats(col1, col2, col3)
print(stats)
```

二、函数返回值与作用域

📚 内容讲解

```
# 返回值
def get_stats(data):
    return {
        'mean': sum(data) / len(data),
        'max': max(data),
        'min': min(data)
    }

# 作用域 - LEGB规则
```

```python
# L: Local（局部）
# E: Enclosing（嵌套）
# G: Global（全局）
# B: Built-in（内置）

x = 10  # 全局变量

def outer():
    y = 20  # 嵌套变量
    def inner():
        z = 30  # 局部变量
        print(x, y, z)
    inner()

outer()  # 10 20 30

# global关键字
count = 0
def increment():
    global count
    count += 1

increment()
print(count)  # 1
```

🎯 应用场景

- 返回字典：返回多个结果
- 返回函数：函数式编程
- 闭包：保存状态

💡 为什么重要

理解作用域避免变量污染，合理使用返回值组织代码。

🔧 实战示例

```python
# 闭包：创建计数器
def create_counter():
    count = 0
    def counter():
        nonlocal count
        count += 1
        return count
    return counter

# 使用
c = create_counter()
print(c())  # 1
print(c())  # 2
print(c())  # 3
```

三、递归函数

📚 内容讲解

```
# 阶乘
def factorial(n):
    if n <= 1:
        return 1
    return n * factorial(n - 1)

# 斐波那契数列
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# 注意：递归深度有限制，Python默认1000
```

🎯 应用场景

- 阶乘、斐波那契
- 遍历树结构
- 分治算法

🔧 实战示例

```
# 递归计算文件夹大小（模拟）
def calculate_size(data):
    if isinstance(data, (int, float)):
        return data
    total = 0
    for item in data:
        total += calculate_size(item)
    return total

# 使用
data = [10, [20, 30], [40, [50, 60]]]
print(calculate_size(data))  # 210
```

四、Lambda表达式

📚 内容讲解

```
# 基本语法
lambda 参数: 表达式

# 示例
add = lambda x, y: x + y
add(3, 5)  # 8
```

🎯 应用场景

- sorted 排序
- map 转换
- filter 筛选

💡 为什么重要

数据分析中经常需要快速转换数据，lambda简洁高效。

🔧 实战示例

```
# 数据分析场景
data = [
    {'name': 'Alice', 'age': 25, 'score': 85},
    {'name': 'Bob', 'age': 20, 'score': 90},
    {'name': 'Charlie', 'age': 30, 'score': 80}
]

# 按分数排序
sorted_data = sorted(data, key=lambda x: x['score'], reverse=True)

# 提取姓名
names = list(map(lambda x: x['name'], data))

# 筛选年龄>25
young_people = list(filter(lambda x: x['age'] > 25, data))

print(sorted_data)
print(names)
print(young_people)
```

五、文件操作

📚 内容讲解

```
# 文本文件
with open('data.txt', 'r', encoding='utf-8') as f:
    content = f.read()

with open('output.txt', 'w', encoding='utf-8') as f:
    f.write('Hello\n')

# CSV文件
import csv
with open('sales.csv', 'r', encoding='utf-8') as f:
    reader = csv.reader(f)
    data = list(reader)

# JSON文件
import json
```

```
data = {'name': '张三', 'age': 25}
with open('data.json', 'w', encoding='utf-8') as f:
    json.dump(data, f, ensure_ascii=False, indent=2)
```

🎯 应用场景

- 读取数据源
- 保存分析结果
- 配置文件

💡 为什么重要

CSV是数据分析标准格式，JSON用于API数据。

🔧 实战示例

```
# 完整的数据分析流程
import csv
import json

def analyze_csv(file_path):
    """读取CSV并分析"""
    with open(file_path, 'r', encoding='utf-8') as f:
        reader = csv.DictReader(f)
        data = list(reader)

    # 提取数值列
    scores = [float(row['score']) for row in data]

    # 计算统计量
    result = {
        'count': len(scores),
        'mean': sum(scores) / len(scores),
        'max': max(scores),
        'min': min(scores)
    }

    return result

# 使用
result = analyze_csv('students.csv')
print(result)

# 保存结果
with open('result.json', 'w', encoding='utf-8') as f:
    json.dump(result, f, ensure_ascii=False, indent=2)
```

六、异常处理

📚 内容讲解

```
try:
    # 可能出错的代码
    num = int(input('输入数字: '))
    result = 10 / num
except ValueError:
    print('请输入数字')
except ZeroDivisionError:
    print('不能除以零')
else:
    print(f'结果: {result}')
finally:
    print('程序结束')
```

🎯 应用场景

- 数据清洗
- 文件读写
- 网络请求

💡 为什么重要

真实数据往往不完美，需要优雅处理异常。

🔧 实战示例

```
# 数据清洗：安全类型转换
def safe_convert(value, target_type):
    try:
        return target_type(value)
    except (ValueError, TypeError):
        return None

# 批量处理
raw_data = ['1.5', '2.0', 'invalid', '3.5', None]
clean_data = [safe_convert(x, float) for x in raw_data]
print(clean_data)  # [1.5, 2.0, None, 3.5, None]
```

七、模块与包

📚 内容讲解

```
# 导入
import math
from math import pi, sqrt
import pandas as pd
```

```python
# 创建模块 data_utils.py
def clean_data(data):
    return [x for x in data if x is not None]

def calculate_stats(data):
    return {'mean': sum(data)/len(data)}

# 使用
from data_utils import clean_data, calculate_stats
```

🎯 应用场景

- 代码组织
- 复用功能
- 使用第三方库

🔧 实战示例

```python
# 文件: data_utils.py
"""数据处理工具模块"""

import csv
import json

def read_csv(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        return list(csv.DictReader(f))

def write_json(data, file_path):
    with open(file_path, 'w', encoding='utf-8') as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

def calculate_mean(numbers):
    clean = [x for x in numbers if x is not None]
    return sum(clean) / len(clean) if clean else None
```

八、迭代器与生成器

📚 内容讲解

```python
# 迭代器
my_list = [1, 2, 3]
it = iter(my_list)
next(it)  # 1

# 生成器函数
def count_down(n):
    while n > 0:
        yield n
        n -= 1

# 生成器表达式
gen = (x * 2 for x in range(1000000))
```

💡 为什么重要

处理大数据时节省内存，不一次性加载所有数据。

🔧 实战示例

```python
# 逐行读取大文件
def read_large_file(file_path):
    with open(file_path, 'r', encoding='utf-8') as f:
        for line in f:
            yield line.strip()

# 统计行数
count = 0
for line in read_large_file('big_data.txt'):
    count += 1
    if count % 10000 == 0:
        print(f'已处理 {count} 行')
```

九、装饰器

📚 内容讲解

```python
import time
from functools import wraps

def timer(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        start = time.time()
        result = func(*args, **kwargs)
        end = time.time()
        print(f'{func.__name__} 耗时: {end - start:.4f}秒')
        return result
    return wrapper

@timer
```

```python
def process_data(data):
    time.sleep(1)
    return sum(data)
```

🎯 应用场景

- 计时
- 缓存
- 日志
- 权限验证

💡 为什么重要

数据分析中经常需要记录执行时间、缓存结果。

下午：面向对象编程（6小时）

十、面向对象基础

📚 内容讲解

```python
class Person:
    # 类属性
    species = '人类'

    # 初始化方法
    def __init__(self, name, age):
        self.name = name  # 实例属性
        self.age = age

    # 方法
    def greet(self):
        print(f'你好，我是{self.name}')

# 创建对象
p1 = Person('张三', 25)
p1.greet()
```

🎯 应用场景

- 封装数据分析逻辑
- 模拟现实概念
- 创建可复用组件

十一、魔术方法

📚 内容讲解

```
class DataContainer:
    def __init__(self, data):
        self.data = data

    def __str__(self):
        return f'DataContainer({self.data})'

    def __len__(self):
        return len(self.data)

    def __getitem__(self, index):
        return self.data[index]

    def __add__(self, other):
        return DataContainer(self.data + other.data)
```

🔧 实战示例

```
dc = DataContainer([1, 2, 3])
print(dc)  # DataContainer([1, 2, 3])
print(len(dc))  # 3
print(dc[0])  # 1

dc2 = DataContainer([4, 5, 6])
dc3 = dc + dc2
print(dc3)  # DataContainer([1, 2, 3, 4, 5, 6])
```

十二、继承与多态

📚 内容讲解

```
class DataAnalyzer:
    def __init__(self, data):
        self.data = data

    def analyze(self):
        raise NotImplementedError('子类必须实现')

class SimpleAnalyzer(DataAnalyzer):
    def analyze(self):
        return {
            'mean': sum(self.data) / len(self.data),
            'count': len(self.data)
        }

class AdvancedAnalyzer(DataAnalyzer):
```

```
    def analyze(self):
        sorted_data = sorted(self.data)
        n = len(sorted_data)
        median = sorted_data[n//2] if n%2 else (sorted_data[n//2-1] + sorted_data[n//2])/2
        return {
            'mean': sum(self.data)/n,
            'median': median,
            'max': max(self.data),
            'min': min(self.data)
        }
```

💡 为什么重要

不同分析器可以统一接口，灵活切换。

十三、实战项目：用类封装数据处理逻辑

🚀 项目目标

创建一个完整的数据分析类，支持：

1. 读取CSV/JSON
2. 数据清洗
3. 统计分析
4. 结果保存

🔧 完整代码

```
import csv
import json
from typing import List, Dict, Any

class DataAnalyzer:
    """数据分析类"""

    def __init__(self):
        self.data = None
        self.cleaned_data = None

    def load_csv(self, file_path: str) -> 'DataAnalyzer':
        """加载CSV文件"""
        with open(file_path, 'r', encoding='utf-8') as f:
            reader = csv.DictReader(f)
            self.data = list(reader)
        return self

    def load_json(self, file_path: str) -> 'DataAnalyzer':
        """加载JSON文件"""
        with open(file_path, 'r', encoding='utf-8') as f:
            self.data = json.load(f)
        return self
```

```python
    def clean_data(self) -> 'DataAnalyzer':
        """清洗数据"""
        if not self.data:
            raise ValueError('请先加载数据')

        self.cleaned_data = []
        for row in self.data:
            cleaned_row = {}
            for key, value in row.items():
                # 尝试转换为数值
                try:
                    if '.' in str(value):
                        cleaned_row[key] = float(value)
                    else:
                        cleaned_row[key] = int(value)
                except (ValueError, TypeError):
                    cleaned_row[key] = value
            self.cleaned_data.append(cleaned_row)
        return self

    def get_numeric_column(self, column_name: str) -> List[float]:
        """获取数值列"""
        if not self.cleaned_data:
            raise ValueError('请先清洗数据')

        values = []
        for row in self.cleaned_data:
            val = row.get(column_name)
            if isinstance(val, (int, float)):
                values.append(val)
        return values

    def analyze_column(self, column_name: str) -> Dict[str, Any]:
        """分析单列"""
        values = self.get_numeric_column(column_name)
        if not values:
            return None

        sorted_values = sorted(values)
        n = len(sorted_values)

        return {
            'column': column_name,
            'count': n,
            'sum': sum(values),
            'mean': sum(values) / n,
            'median': sorted_values[n//2] if n%2 else (sorted_values[n//2-1] +
sorted_values[n//2])/2,
            'min': min(values),
            'max': max(values)
        }

    def save_result(self, result: Dict[str, Any], file_path: str) -> 'DataAnalyzer':
        """保存结果"""
        with open(file_path, 'w', encoding='utf-8') as f:
            json.dump(result, f, ensure_ascii=False, indent=2)
        return self
```

```python
    def __str__(self):
        if self.data:
            return f'DataAnalyzer({len(self.data)} 条数据)'
        return 'DataAnalyzer(空)'


# ==================== 使用示例 ====================

if __name__ == '__main__':
    # 1. 创建分析器
    analyzer = DataAnalyzer()

    # 2. 加载数据
    analyzer.load_csv('students.csv')
    print(analyzer)

    # 3. 清洗数据
    analyzer.clean_data()

    # 4. 分析分数列
    result = analyzer.analyze_column('score')
    print('分析结果:', result)

    # 5. 保存结果
    analyzer.save_result(result, 'analysis_result.json')
    print('结果已保存')
```

📝 配套练习

练习1：函数参数

1. 写一个函数，支持任意数量的数值参数，返回它们的平均值
2. 写一个函数，接受名字、年龄、城市，城市有默认值"未知"

练习2：Lambda与高阶函数

1. 用 `sorted` 和 lambda 按字典的 `score` 字段排序
2. 用 `map` 将数字列表转为字符串列表
3. 用 `filter` 筛选出大于10的数

练习3：文件操作

1. 读取一个CSV文件，计算某列的平均值
2. 将一个字典列表写入JSON文件

练习4：异常处理

1. 写一个 `safe_divide` 函数，安全处理除零错误
2. 写一个 `safe_int` 函数，转换失败返回None

练习5：面向对象

1. 创建一个 `Student` 类，有姓名、年龄、分数属性
2. 创建 `Classroom` 类，包含多个学生，有计算平均分的方法
3. 给 `Classroom` 添加魔术方法 `__len__` 和 `__str__`

✅ 自我检测清单

- √ 理解位置参数、关键字参数、默认参数、可变参数的区别
- √ 能写出接受任意数量参数的函数
- √ 理解LEGB作用域规则
- √ 能用lambda配合sorted/map/filter处理数据
- √ 能读写文本、CSV、JSON三种格式的文件
- √ 会用try-except处理异常
- √ 能创建自己的模块并导入使用
- √ 理解生成器和迭代器，能写简单的生成器
- √ 能写至少一个实用的装饰器（如计时）
- √ 能创建类，使用 `__init__`、`__str__` 等魔术方法
- √ 理解继承和多态的基本概念
- √ 能独立完成下午的实战项目（DataAnalyzer类）

如果你能完成实战项目，说明你已经掌握了今天的内容！ 🎉

📚 推荐资源

- Python官方文档
- 《Python编程：从入门到实践》

- 菜鸟教程 Python3 教程